

SecurAx Scanner Engine

Couverture	DAST, SAST, Réseau, SSL, Dépendances, DNS, Docker, Web, WP, Serveurs
Sujet	Périmètre d'action, limites techniques (Faux Positifs/Négatifs) et heuristiques
Langue	Français
Date	Juin 2026

1. SCANNER DAST (dast_scanner.py)

1.1 Capacités et Périmètre d'Action

Moteur d'analyse dynamique des applications web (Dynamic Application Security Testing) utilisant l'API OWASP ZAP.

Fonctionnalité	Détail de l'analyse
Spidering Actif	Cartographie récursive du site pour découvrir toutes les URLs, formulaires et paramètres cachés.
Injection Dynamique	Envoi de payloads malveillants réels (SQLi, XSS, Command Injection) dans tous les inputs découverts.
Analyse d'État	Surveille les réponses HTTP pour détecter si l'injection a modifié le comportement du serveur.

1.2 Méthodologie et Outils

1	Orchestration API REST	Le scanner Python ne fait pas l'attaque lui-même : il pilote un démon OWASP ZAP via des requêtes HTTP (démarrage du spider, lancement de l'active scan, polling du statut).
2	Heuristique Comportementale	Évalue la vulnérabilité non pas en lisant le code, mais en analysant les erreurs SQL ou les alertes Javascript renvoyées dans le corps HTTP.

1.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Temps d'exécution massif	Un scan complet génère des milliers de requêtes. Il peut prendre des heures sur un gros site et causer un déni de service (DoS) local.
Authentification Complexe	Incapable de scanner au-delà des pages de connexion complexes (MFA, Captcha) sans configuration manuelle lourde de sessions.
Pas de visibilité interne	Signale qu'une SQLi existe, mais ne peut pas indiquer à quelle ligne de code elle se trouve (boîte noire).

2. SCANNER SAST / CODE AUDIT (sast_scanner.py)

2.1 Capacités et Périmètre d'Action

Analyse statique du code source Python (Static Application Security Testing) sans l'exécuter.

Fonctionnalité	Détail de l'analyse
Détection de Secrets	Repère les mots de passe, tokens et clés API hardcodés dans les fichiers .py.
Code Dangereux	Identifie l'utilisation de <code>`eval()`</code> , <code>`exec()`</code> , <code>`pickle`</code> (désérialisation unsafe) et <code>`subprocess`</code> avec <code>shell=True</code> .
Injectons SQL Statiques	Détecte la concaténation de chaînes dans les requêtes SQL (au lieu de la paramétrisation).

2.2 Méthodologie et Outils

1	Arbre Syntaxique Abstrait (AST)	Le module enveloppe l'outil 'Bandit'. Bandit compile le code source Python en AST, puis parcourt les nœuds pour identifier les patterns vulnérables, évitant ainsi les erreurs des simples regex.
2	Sous-processus Isolés	Le scanner exécute Bandit dans un subprocess sécurisé et parse sa sortie JSON de manière asynchrone.

2.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Faux Positifs Fréquents	S'alarme souvent sur du code de test ou des variables nommées 'password' même si elles sont sécurisées.
Limitation au Langage Python	Ne scanne pas le code Javascript (React) ou PHP, il est strictement limité aux fichiers Python.
Ignorance du Runtime	Une faille identifiée statiquement n'est pas forcément exploitable en réalité si un WAF bloque l'entrée avant qu'elle n'atteigne la fonction.

3. SCANNER RÉSEAU ET PORTS (netscan_scanner.py)

3.1 Capacités et Périmètre d'Action

Cartographie du réseau et découverte de la surface d'attaque externe de l'infrastructure.

Fonctionnalité	Détail de l'analyse
Découverte de Ports	Scanne les 1000 ports TCP les plus courants ou l'ensemble des 65535 ports pour trouver les services ouverts.
Détection de Version (Fingerprinting)	Interroge les bannières de service pour déterminer la version exacte du logiciel tournant sur le port (ex: OpenSSH 8.2p1).
Vulnérabilités OS	Peut exécuter des scripts NSE (Nmap Scripting Engine) pour détecter des vulnérabilités célèbres (ex: MS17-010 EternalBlue).

3.2 Méthodologie et Outils

1	Wrapper python-nmap	Utilise la librairie `python-nmap` pour interfacer le binaire système de Nmap. Traduit les résultats bruts et le XML en dictionnaires Python.
2	Techniques TCP SYN	Utilise par défaut le scan TCP SYN (Half-open) pour être discret et rapide, ne terminant pas le 3-way handshake.

3.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Bloqué par les Pare-feu (Firewalls)	Les IDS/IPS et Firewalls détectent rapidement les scans de ports agressifs et bannissent l'adresse IP du scanner.
Pas d'analyse applicative	Il sait que le port 80 est ouvert et fait tourner Apache, mais ne sait pas si l'application PHP hébergée dessus contient une faille XSS.

4. SCANNER SSL / TLS (ssl_scanner.py)

4.1 Capacités et Périmètre d'Action

Audit cryptographique de la couche de transport sécurisée (HTTPS/TLS).

Fonctionnalité	Détail de l'analyse
Expiration des Certificats	Alerte sur les certificats expirés ou sur le point d'expirer (moins de 30 jours).
Protocoles Obsolètes	Détecte si le serveur supporte encore SSLv2, SSLv3, TLS 1.0 ou TLS 1.1 (interdits par les standards modernes).
Algorithmes Faibles	Vérifie si des ciphers obsolètes (RC4, DES, 3DES) ou sans Forward Secrecy sont utilisés.

4.2 Méthodologie et Outils

1	Socket SSL Natif	Utilise le module Python `ssl` et `socket` pour négocier la connexion (Handshake) au port 443 et inspecter le certificat renvoyé par le serveur cible sans dépendance externe.
2	Parsing x509	Extrait les dates 'notBefore' et 'notAfter', l'émetteur (Issuer), et le Subject Alternative Name (SAN).

4.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Analyse de code impossible	L'audit SSL est purement réseau. Il ne détecte pas si la bibliothèque OpenSSL du serveur est vulnérable à Heartbleed (nécessite Nmap/NSE pour cela).
Ne vérifie pas la chaîne de confiance	N'effectue pas une validation stricte complète de l'autorité de certification racine (CA) locale de l'OS hôte.

5. SCANNER DE DÉPENDANCES (dep_scanner.py)

5.1 Capacités et Périmètre d'Action

Analyse de la Supply Chain (SCA - Software Composition Analysis).

Fonctionnalité	Détail de l'analyse
Inventaire SBOM	Dresse la liste exacte des paquets tiers utilisés (ex: requirements.txt ou package.json).
Corrélation CVE	Croise les versions trouvées avec des bases de vulnérabilités connues.
Supply Chain Attack	Détecte les composants abandonnés ou compromis (Typosquatting potentiel).

5.2 Méthodologie et Outils

1	API OSV.dev / PyPI	Interroge des bases de données de vulnérabilités publiques (Open Source Vulnerabilities) via API HTTP avec la liste des dépendances.
2	Parsing de Fichiers de Manifeste	Analyse syntaxique des fichiers de verrouillage (package-lock.json, Pipfile.lock) pour extraire l'arbre exact des versions.

5.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Ignorance de l'utilisation réelle	Il signale que la librairie 'requests' 2.20 est vulnérable, mais ne sait pas si votre code appelle réellement la fonction vulnérable de cette librairie (Faux Positifs fréquents).
Base de données dépendante	Si la vulnérabilité est un 0-day et n'est pas encore documentée dans l'OSV, le scanner ne verra absolument rien.

6. SCANNER WEB DE BASE (web_scanner.py)

6.1 Capacités et Périmètre d'Action

Audit de surface HTTP rapide et évaluation des en-têtes de sécurité.

Fonctionnalité	Détail de l'analyse
Security Headers	Vérifie la présence de CSP, HSTS, X-Frame-Options, X-Content-Type-Options.
Fuites d'Informations	Détecte l'en-tête 'Server' (ex: Apache/2.4.41) et 'X-Powered-By' (ex: PHP/7.4).
Redirections Insecure	S'assure que le trafic HTTP est forcé vers HTTPS en analysant les codes 301/302.

6.2 Méthodologie et Outils

1	Requêtes HTTP Pures	Utilise la librairie standard 'requests' pour faire une requête GET sur la racine du site et inspecte le dictionnaire des headers (r.headers).
---	---------------------	--

6.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Pas de Crawling	Il ne visite que l'URL fournie (généralement la page d'accueil). Il ne découvrira pas qu'une page cachée '/admin' manque de headers.
Pas d'Injection	Il n'envoie aucun payload malveillant. C'est un outil purement passif et défensif.

7. AUDIT DE CONFIGURATION SERVEUR (server_int.py)

7.1 Capacités et Périmètre d'Action

Analyse statique des fichiers de configuration système (Apache/Nginx/SSH).

Fonctionnalité	Détail de l'analyse
Exposition des Versions	Vérifie si 'ServerTokens Prod' ou 'server_tokens off' sont bien configurés.
SSL/TLS Serveur	Analyse 'SSLProtocol' pour s'assurer que TLSv1.2/1.3 sont forcés sur le serveur web.
Accès Racine SSH	Analyse sshd_config pour vérifier 'PermitRootLogin no' et 'PasswordAuthentication no'.

7.2 Méthodologie et Outils

1	Expressions Régulières Statiques	Charge les fichiers texte (comme nginx.conf) et utilise des Regex pour chercher des mauvaises pratiques de configuration courantes.
---	----------------------------------	---

7.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Nécessite un accès fichier local	Ce scanner ne peut pas fonctionner à distance. Il doit avoir accès au système de fichiers de l'hôte pour lire les .conf (Idéal pour des audits Blue Team).
Ignorance des surcharges	Il peut trouver une faille dans le fichier principal, mais ignorer qu'elle est corrigée dans un fichier 'include conf.d/*' qu'il ne parse pas.

8. SONDAGE SERVEUR EXTERNE (server_ext.py)

8.1 Capacités et Périmètre d'Action

Évaluation rapide de l'empreinte externe et de la résolution DNS sans balayage agressif.

Fonctionnalité	Détail de l'analyse
Empreinte (Fingerprinting)	Identifie rapidement le type de serveur web via les requêtes HEAD.
Geo-IP et Hébergeur	Identifie l'ASN et le Cloud Provider hébergeant l'IP.
WAF Detection	Tente de détecter la présence d'un pare-feu applicatif (Cloudflare, AWS WAF).

8.2 Méthodologie et Outils

1	Requêtes HEAD légères	Utilise des requêtes HTTP HEAD (plutôt que GET) pour minimiser la consommation de bande passante.
2	Bases de données ASN	Effectue des requêtes WHOIS sur l'IP pour déterminer l'appartenance de l'infrastructure.

8.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Superficiel	Ce n'est pas un scanner de vulnérabilités, mais un outil de reconnaissance (Recon) initiale.
Facilement trompé	Un administrateur peut modifier l'en-tête 'Server' pour afficher 'Commodore64' au lieu de 'Nginx', trompant totalement le scanner.

9. SCANNER DNS & EMAIL SECURITY (dns_scanner.py)

9.1 Capacités et Périmètre d'Action

Vérifie la robustesse de l'infrastructure DNS et la lutte contre le phishing.

Fonctionnalité	Détail de l'analyse
Prévention Spoofing	Valide les politiques SPF, DMARC et DKIM pour empêcher l'envoi d'emails frauduleux.
Intégration Cryptographique	Vérifie l'existence des signatures DNSSEC et les enregistrements CAA.
Fuite de Zone	Tente d'exécuter un Zone Transfer (AXFR) pour télécharger toute la configuration réseau interne du domaine.

9.2 Méthodologie et Outils

1	DNS-over-HTTPS (DoH)	Utilise l'API Cloudflare DNS via HTTP pour contourner les blocages du port 53 en environnement d'entreprise restreint.
2	Sockets TCP purs	Pour tester l'AXFR, il ouvre un socket bas niveau et construit manuellement les bits d'une requête DNS de type Zone Transfer.

9.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Limite inhérente au DKIM	Impossible de découvrir toutes les clés DKIM sans connaître le nom exact du 'sélecteur' utilisé par l'entreprise (limite du protocole DNS).
Limites des redirections SPF	Ne suit pas toujours les chaînes infinies d'inclusions SPF pour valider la limite stricte de 10 résolutions du RFC.

10. SCANNER CONTENEURS & DOCKER (docker_scanner.py)

10.1 Capacités et Périmètre d'Action

Audit statique de la sécurité des conteneurs via Dockerfile et docker-compose.

Fonctionnalité	Détail de l'analyse
Isolation des Privilèges	Alerte sur les conteneurs exécutés en root ('USER root' ou omission du directive USER).
Exposition dangereuse	Signale le montage du socket docker ('/var/run/docker.sock') et le mode 'privileged: true'.
Analyse d'ENV	Cherche les clés d'API et mots de passe codés en dur dans les variables d'environnement (ENV).

10.2 Méthodologie et Outils

1	Parsing Ligne par Ligne	Il n'exécute pas le démon Docker. Il charge le texte des fichiers de configuration, sépare les instructions (FROM, RUN, ENV, CMD) et applique des règles de sécurité (Regex).
---	--------------------------------	---

10.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Pas de scan d'images compilées	Il ne peut pas vérifier le système de fichiers réel à l'intérieur de l'image (ex: si des binaires SUID malveillants y ont été placés).
Faux positifs sur les variables	Peut signaler des variables d'environnement comme secrètes alors qu'elles sont simplement définies comme des templates (ex: ENV DB_PASS=\$SECRET).

11. SCANNER WORDPRESS

(wordpress_scanner.py)

11.1 Capacités et Périmètre d'Action

Probing spécifique aux CMS WordPress pour détecter les erreurs de configuration courantes.

Fonctionnalité	Détail de l'analyse
Version Disclosure	Trouve la version exacte (via readme.html ou meta generator) et vérifie si elle est obsolète/vulnérable.
Énumération d'Utilisateurs	Interroge l'API REST (/wp-json/wp/v2/users) pour lister les comptes et vérifier la présence du login par défaut 'admin'.
Vecteurs d'attaque	Vérifie si xmlrpc.php (DDoS/SSRF) et wp-cron.php (DoS) sont accessibles publiquement.
Fichiers Sensibles	Tente de télécharger /wp-config.php et /wp-content/debug.log.

11.2 Méthodologie et Outils

1	Requêtes HTTP Ciblées	Envoie des requêtes vers des chemins standards de WordPress. Ce n'est pas un Fuzzer aveugle, il sait exactement où chercher.
2	JSON Parsing	Exploite l'API REST ouverte par défaut depuis WP 4.7 pour extraire proprement les listes d'auteurs au format JSON.

11.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Ignorance des Plugins (par défaut)	Il ne contient pas de base de données gigantesque des vulnérabilités de plugins tiers (contrairement à WPScan). Il se concentre sur le cœur (Core) de WordPress.
Facilement bloqué par les WAF	N'importe quel plugin de sécurité WordPress basique (Wordfence) bloquera instantanément ces tentatives rudimentaires, rendant le scanner aveugle.